

Rim Defect Detection

TPU Notebook Guide

What changed between the GPU notebook (v4.0) and the TPU version, explained clearly for every section — even if you have never used a TPU before.

Notebook: [tpu-notebook-computervision.ipynb](#)
Reference: [computer-vision-notebook_v4_0.ipynb](#)
Updated: May 2026 | 12 Change Sections

Table of Contents

Section 1	— Overview — What Changed and Why?
Section 2	— Section 1: Environment Setup — torch_xla Replaces CUDA
Section 3	— Section 2: Configuration — Batch Size, Accumulation, Precision
Section 4	— Section 3: Stage 1 Training — Bigger Batches, Native bfloat16
Section 5	— Section 4: Data Loading — DistributedSampler & No pin_memory
Section 6	— Section 5: Augmentations — Two New Transforms Added
Section 7	— Section 6: Rim Cropping — HoughCircles Replaces Fixed Coordinates
Section 8	— Section 7: Training Loop — xm.* API Replaces torch.cuda.*
Section 9	— Section 8: Validation & BN Recalibration — MpDeviceLoader
Section 10	— Section 9: Inference Pipeline — Grad-CAM Removed
Section 11	— Section 10: Multi-Process Entry Point — xmp.spawn
Section 12	— Complete Side-by-Side Hyperparameter Reference

1. Overview — What Changed and Why?

The GPU notebook (v4.0) was designed to run on a single NVIDIA GPU — a T4 or A100 on Kaggle. The TPU notebook rewrites the same algorithm to run on a Kaggle **TPU v5e-8**, which is a board containing **8 independent TPU chips**, each with 16 GB of HBM memory.

What is a TPU?

A TPU (Tensor Processing Unit) is Google's custom chip for deep learning. Unlike a GPU, which executes arbitrary CUDA code, a TPU runs a **compiled graph** of matrix operations. This has two big implications for the code:

1. You cannot use standard **torch.cuda.*** calls — everything goes through **torch_xla** (XLA = Accelerated Linear Algebra), Google's compiler bridge between PyTorch and the TPU hardware.
2. Operations are **lazy** — they are queued up in an XLA graph and only actually executed when you call **xm.mark_step()**. Forgetting **mark_step()** is the #1 bug when porting GPU code to TPU.

The 10 Core Changes at a Glance

Parameter / Feature	GPU Notebook (v4.0)	TPU Notebook
Compute backend	torch.cuda / CUDA	torch_xla / XLA
Device creation	torch.device('cuda')	xm.xla_device()
Batch size	2 (GPU notebook default)	192 (TPU; 8 chips × 24)
Gradient accum.	32 steps (simulates large batch)	1 step (large batch is real)
Precision	bfloat16 via torch.autocast('cuda', ...)	bfloat16 native; torch.autocast('xla', ...)
Loss scaling	GradScaler for float16	Removed — bfloat16 needs no scaler
Optimizer step	optimizer.step()	xm.optimizer_step(optimizer)
Data prefetch	DataLoader pin_memory=True	pl.MpDeviceLoader wraps each loader
Multi-chip	Single GPU, single process	xmp.spawn — one process per chip
Grad-CAM crop	Dynamic backward-hook heatmap	Removed; HoughCircles crop used instead
Channels-last	True (GPU NCHW optimisation)	False (not applicable on TPU)
torch.compile	Optional flag (experimental)	Disabled — not supported on TPU XLA
Gradient ckpt	True (saves VRAM on GPU)	False (16 GB/chip is enough at batch 192)

2. Environment Setup — torch_xla Replaces CUDA

GPU Version (v4.0)

The GPU notebook imports PyTorch and sets the CUDA memory allocator config. Device is selected with `torch.device('cuda' if torch.cuda.is_available() else 'cpu')`. No special installation is needed because CUDA drivers come with Kaggle's GPU images.

```
os.environ['PYTORCH_CUDA_ALLOC_CONF'] = 'max_split_size_mb:64'
```

TPU Version — What Changed

The TPU notebook **removes** the CUDA memory config (irrelevant on TPU) and adds a cell that installs **torch_xla** if it is missing, then imports the full XLA API:

```
import torch_xla
import torch_xla.core.xla_model as xm
import torch_xla.runtime as xr
import torch_xla.distributed.xla_multiprocessing as xmp
import torch_xla.distributed.parallel_loader as pl
```

Two new helper functions

get_device() — returns `xm.xla_device()` instead of `torch.device('cuda')`. Each of the 8 spawned worker processes calls this independently and gets its own chip.

get_num_tpu_chips() — queries `xr.world_size()` to find out how many chips are available (8 on a v5e-8). This drives the `xmp.spawn` call later.

```
def get_device(): return xm.xla_device() def get_num_tpu_chips(): try: return xr.world_size() #
torch_xla >= 2.1 except Exception: return 8
```

Note: `scipy` is also imported in the TPU version (from `scipy.optimize` import `minimize`) for potential future use in threshold optimisation. It was not present in v4.0.

3. Configuration — Batch Size, Accumulation, Precision

The argument parser is almost identical between versions, but several defaults change dramatically because the TPU's large memory and 8-chip setup changes what is optimal.

Parameter / Feature	GPU Notebook (v4.0)	TPU Notebook
<code>--batch</code>	2 (GPU, memory-limited)	192 (TPU, 8 chips × ~24 each)
<code>--accum-steps</code>	32 (simulates batch ~64)	1 (real batch 192 needs no accumulation)
<code>--img-size</code>	384 (override from 256 for Swin)	384 (same)
<code>--label-smooth</code>	0.02	0.01 (tighter — large batches already regularise)
<code>--mixup-alpha</code>	0.10	0.20 (more aggressive MixUp)
<code>--focal-gamma</code>	2.0	2.5 (harder focus on difficult examples)
<code>--ema-decay</code>	0.9998	0.9999 (slower EMA — more stable at large batch)
<code>--bf16</code>	False (GPU needs explicit opt-in)	True (TPU native, always on)
<code>--channels-last</code>	True (NCHW GPU optimisation)	False (not applicable on TPU)
<code>--compile</code>	False (optional flag)	False (torch.compile unsupported on XLA)
<code>--grad-checkpoint</code>	True (saves GPU VRAM)	False (16 GB/chip is sufficient)
<code>STAGE1_BATCH</code>	8	128 (TPU handles larger batches easily)

Why does batch size jump from 2 to 192?

In the GPU notebook, each image at 384×384 with a 3-backbone ensemble consumes roughly 5-6 GB of GPU VRAM. A single T4 has 16 GB total, leaving room for only 2 images per batch before running out of memory. To simulate the effect of a larger batch (which stabilises gradient updates), the GPU version accumulates gradients over 32 steps before updating weights — so the effective batch size is $2 \times 32 = 64$.

A TPU v5e-8 has 8 chips, each with 16 GB of HBM. At batch size 192 across 8 chips, each chip sees $192 / 8 = 24$ images per step. Memory per chip is well within limits, so gradient accumulation is no longer needed. `accum-steps` is set to 1.

Why does a larger real batch need a slower EMA (0.9999 vs 0.9998)? Because each gradient step is computed on more data and is therefore more reliable. The EMA should move more slowly to capture the long-run average rather than tracking noisy individual steps.

4. Stage 1 Training — Bigger Batches, Native bfloat16

Stage 1 trains the lightweight EfficientNet-B0 to quickly flag obvious defects before the expensive ensemble runs. The architecture and loss function are **identical** between versions. Only the execution layer changes.

Changes in Stage 1

Batch size: Raised from 8 to 128. On a single TPU chip this is easily achievable at 384px.

pin_memory=False: On a GPU, `pin_memory=True` speeds up host-to-device transfers by using pinned (page-locked) CPU memory. TPU memory management is handled by XLA internally — pinned memory is not a concept on TPU and the flag is always False.

autocast backend: Changed from 'cuda' to 'xla'.

```
torch.autocast('cuda', dtype=torch.bfloat16) # GPU version
torch.autocast('xla', dtype=torch.bfloat16) # TPU version
```

Optimizer step: Changed from `optimizer.step()` to `xm.optimizer_step(optimizer)`. This is critical — `xm.optimizer_step()` does the weight update AND calls `xm.mark_step()` internally, flushing the pending XLA computation graph. Forgetting this causes the training to silently do nothing on TPU.

Checkpoint saving: Changed from `torch.save()` to `xm.save()`. This is required because XLA tensors live on the TPU chip's HBM memory, not regular CPU RAM. `xm.save()` handles the device-to-host transfer before pickling.

xm.mark_step() after EMA update: After updating EMA weights (which are also XLA tensors), an explicit `mark_step()` flushes the graph so the next iteration starts clean.

Warning: In Stage 1 on the GPU version, there is a known bug where `pos_weight` was set to `~9.0` even though the training set was already oversampled to 1:1. Both versions fix this by setting `pos_weight = 1.0` when oversampling is enabled.

5. Data Loading — DistributedSampler & No pin_memory

When 8 TPU chips each run a copy of the training loop (via `xmp.spawn`), they must not all train on the same data. Each chip should train on a disjoint 1/8th of the dataset — otherwise you are wasting 7/8 of your compute.

DistributedSampler

The GPU notebook uses a simple `WeightedRandomSampler` or `shuffle=True`. The TPU notebook adds a `DistributedSampler` that automatically divides the dataset among chips:

```
train_sampler = DistributedSampler( train_dataset, num_replicas=world_size, # 8 chips
rank=rank, # this chip's index (0-7) shuffle=True, )
```

Each chip (rank 0 through 7) sees a unique subset of the training data. After all 8 chips finish a batch, `xm.optimizer_step()` all-reduces the gradients (averages them across chips) before updating weights. This is equivalent to training on the full batch at once.

MpDeviceLoader

Instead of iterating the `DataLoader` directly, every loop in the TPU notebook wraps it with `pl.MpDeviceLoader`:

```
xla_loader = pl.MpDeviceLoader(loader, device) for imgs, lbls in xla_loader: ... # imgs is
already on the XLA device
```

`MpDeviceLoader` prefetches the next batch to the XLA device asynchronously while the current batch is being computed. This hides the host-to-device transfer latency, keeping the TPU MXUs (matrix multiply units) busy instead of waiting for data.

Parameter / Feature	GPU Notebook (v4.0)	TPU Notebook
pin_memory	True (speeds up GPU host-device transfers)	False (XLA manages its own memory)
Shuffle	<code>shuffle=True</code> in <code>DataLoader</code>	<code>DistributedSampler(shuffle=True)</code>
Sampler	<code>WeightedRandomSampler</code> or none	<code>DistributedSampler</code> for multi-chip
Device loader	<code>DataLoader</code> iterated directly	<code>pl.MpDeviceLoader</code> wraps the loader
persistent_workers	True (keep workers alive)	False (XLA handles prefetch)

6. Augmentations — Two New Transforms Added

All five custom augmentation classes from v4.0 are preserved unchanged in the TPU notebook (RadialBlur, SpecularHighlight, ScratchSimulation, SectorMask, RimSpecificAug). They all run on CPU before tensors are sent to the device — which is the correct pattern for both GPU and TPU.

What is new in the TPU version

The TPU notebook adds **two augmentations** to the heavy training pipeline that were absent in v4.0:

A.GridDistortion(p=0.25) — Applies a local grid-based distortion to the image. Imagine bending a photograph on a rubber sheet. This simulates lens distortion and makes the model robust to slight geometric warping from camera optics.

A.RandomShadow(p=0.20) — Adds a random dark polygon to part of the image, simulating shadows from factory lighting rigs, camera mounts, or the rim fixture itself.

Why add them now?

With batch size 192 and all layers unfrozen in Phase 3, the model sees far more gradient signal per epoch than the GPU version did. More aggressive augmentation prevents it from overfitting on the larger effective dataset the big batch presents each update.

Note: All augmentations still run on CPU. The comment in the notebook states this explicitly: 'keep MXUs busy with matmuls, not image ops'. The TPU's matrix multiply units are optimised for dense linear algebra — offloading image transforms to CPU workers keeps the pipeline efficient.

7. Rim Cropping — HoughCircles Replaces Fixed Coordinates

GPU Version (v4.0) — Fixed Relative Coordinates

In v4.0, the rim crop was computed from hard-coded relative coordinates derived from the camera's fixed position (images always 1280×1024):

```
rel_cx = (364 + 526/2) / 1280.0 rel_cy = (247 + 526/2) / 1024.0 rel_side = 526.0 / min(1280.0, 1024.0)
```

This works well when every image comes from the same camera at the same position. The fix in v4.0 made these relative (not absolute pixels) so it adapts to different resolutions. However, the crop is still centred at a fixed point — if the rim is off-centre, the crop will be off too.

TPU Version — Adaptive HoughCircles Crop

The TPU notebook replaces the fixed-coordinate approach with an **adaptive Hough Circle detector** that finds the actual rim circle in the image before cropping:

```
gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY) blurred = cv2.GaussianBlur(gray, (9, 9), 2)
circles = cv2.HoughCircles(blurred, cv2.HOUGH_GRADIENT, dp=1.2, minDist=min(h,w)//2,
param1=100, param2=30, minRadius=int(min(h,w)*0.15), maxRadius=int(min(h,w)*0.55)) if circles
is not None: cx, cy, r = circles[0, 0].astype(int) side = int(r * 2 * (1 + margin)) else: cx, cy
= w // 2, h // 2 # fallback to centre side = int(min(h,w) * 0.55 * 2 * (1 + margin))
```

How HoughCircles works

HoughCircles is a classical computer vision algorithm. It scans the edge image for circular patterns using the Hough Transform — essentially asking 'which circle, if drawn here with radius r, would pass through the most detected edges?' The parameters are:

Parameter / Feature	TPU Notebook
dp=1.2	Resolution ratio for the accumulator grid. 1.2 = slightly lower resolution than input, faster.
minDist	Minimum distance between detected circle centres. Set to half the image — only one rim expected.
param1=100	Upper Canny edge threshold. Higher = only strong edges count.

Parameter / Feature	TPU Notebook (v4.0)	TPU Notebook
param2=30	—	Accumulator threshold. Lower = finds more circles (more false positives). 30 is moderate.
minRadius	—	Smallest rim radius to look for (15% of image short side).
maxRadius	—	Largest rim radius (55% of image short side).

If HoughCircles fails to find a circle (e.g., on a very unusual image), the code falls back to the same centre-based crop as v4.0. The margin parameter is 0.10 (10% border around the detected circle) rather than 0.05 in v4.0.

Why is Grad-CAM Dynamic Cropping gone?

v4.0 introduced a Grad-CAM based crop where Stage 1 (B0) identified the suspected defect hotspot and the ensemble received a zoomed-in patch of that area. Grad-CAM requires a backward pass through the model at inference time — which means calling `loss.backward()` outside of a training step. On XLA/TPU, dynamic backward hooks are difficult to implement and can break graph tracing. The TPU notebook removes Grad-CAM entirely and uses the simpler HoughCircles crop instead. This is a deliberate trade-off: slightly less targeted cropping in exchange for clean, traceable XLA graphs.

8. Training Loop — xm.* API Replaces torch.cuda.*

The three-phase training structure (head warm-up, partial unfreeze, full fine-tune) is identical. Loss functions, EMA, MixUp/CutMix, and SWA are all preserved. The changes are all in how PyTorch interacts with the hardware.

Key substitutions inside the batch loop

Parameter / Feature	GPU Notebook (v4.0)	TPU Notebook
torch.autocast	<code>torch.autocast('cuda', dtype=...)</code>	<code>torch.autocast('xla', dtype=torch.bfloat16)</code>
GradScaler	<code>scaler = GradScaler(); scaler.scale(loss).backward(); scaler.step(opt)</code>	Removed. bfloat16 on TPU needs no scaling.
Optimizer step	<code>optimizer.step()</code>	<code>xm.optimizer_step(optimizer)</code>
After EMA update	— (nothing needed)	<code>xm.mark_step()</code> to flush XLA graph
Memory report	<code>torch.cuda.memory_allocated()</code>	<code>xm.get_memory_info(device)</code>
Checkpoint save	<code>torch.save(ckpt, path)</code>	<code>xm.save(ckpt, path)</code>
Master rank check	— (single process)	<code>xm.is_master_ordinal()</code> guards prints/saves
Early stop sync	— (single process)	<code>xm.mesh_reduce('key', val, fn)</code> broadcasts across chips

xm.mark_step() — the most important new call

XLA compiles and runs operations lazily. You can define a thousand operations, but none of them actually execute until you call `xm.mark_step()`. This is called **deferred execution**. The benefit is that XLA can see the entire computation graph and optimise it (fuse operations, reorder for cache efficiency, etc.) before running anything.

The cost is that you must call `mark_step()` at the right moments. If you never call it, the graph grows forever and you run out of memory. The TPU notebook calls it:

- After each training batch (via `xm.optimizer_step`, which includes `mark_step` internally)
- After the EMA update (explicitly)
- After each validation batch
- After each BN recalibration batch
- During inference at each batch

xm.mesh_reduce — Distributed Early Stopping

In the GPU version, early stopping is simple: if the validation F1 has not improved for N epochs, stop. With 8 chips each running independently, each chip has its own early stopping counter. If chip 0 wants to stop but chip 7 does not, you have a deadlock. `xm.mesh_reduce` solves this by broadcasting a value across all chips and applying a reduction function:

```
do_break = xm.mesh_reduce('early_stop', do_break, lambda xs: any(xs))
```

If ANY chip decides it is time to stop (`do_break=True`), all chips stop. Similarly, `epochs_no_improve` is synchronised with `max()` across chips so all chips agree on the current patience counter.

9. Validation & BN Recalibration — MpDeviceLoader

The validation logic (`_validate`, `_sweep_threshold`, `_recalibrate_bn`) is structurally identical to v4.0. The only changes are using `pl.MpDeviceLoader` and calling `xm.mark_step()` after each batch.

`_validate` (TPU changes)

The validation loader is wrapped in `MpDeviceLoader` so batches are prefetched to the XLA device asynchronously. After computing predictions for each batch, `xm.mark_step()` flushes the graph and moves results to CPU for metric calculation:

```
xla_loader = pl.MpDeviceLoader(loader, device) for imgs, lbls in xla_loader: logits, _ =  
model(imgs) probs = torch.sigmoid(logits) xm.mark_step() # flush XLA graph  
all_probs.append(probs.cpu())
```

Threshold sweep runs on CPU

The threshold sweep (testing F1 at every threshold from 0.05 to 0.95) converts probabilities to CPU tensors first, then does the vectorised comparison. This is appropriate — the sweep involves small tensors and simple arithmetic, so running it on CPU avoids unnecessary XLA graph compilation overhead.

`_recalibrate_bn` (TPU changes)

Same approach: wrap the loader with `MpDeviceLoader`, run 25 batches to update BatchNorm statistics, call `xm.mark_step()` after each batch. The `model.train() / model.eval()` pattern and the `momentum=None` trick are unchanged.

10. Inference Pipeline — Grad-CAM Removed

The cascade inference pipeline (Stage 1 fast filter → Stage 2 ensemble) is preserved. The two-stage logic is identical. What changes is the crop method and the absence of Grad-CAM.

Parameter / Feature	GPU Notebook (v4.0)	TPU Notebook
Stage 1 threshold	0.70 (same)	0.70 (unchanged)
Stage 1 input size	384px (same)	384px (unchanged)
Rim crop method	Fixed relative coordinates + optional Grad-CAM zoom	HoughCircles adaptive crop; Grad-CAM removed
TTA	8 D4 views (preserved)	8 D4 views (preserved)
Platt calibration	Loaded from .pkl file	Loaded from .pkl file (unchanged)
mark_step in loop	— (not needed on GPU)	xm.mark_step() after each inference batch
Save checkpoint	torch.save()	xm.save() (handles XLA tensor transfer)

Why no Grad-CAM on TPU?

Grad-CAM works by registering a backward hook on the last convolutional layer, running a forward pass, calling `loss.backward()` to compute gradients, and then using those gradients to build a spatial heatmap. The problem on XLA/TPU is that:

1. XLA traces the computation graph statically. Dynamic backward hooks introduce control flow that XLA cannot trace ahead of time.
2. Calling `loss.backward()` at inference time (inside `torch.no_grad()`) requires temporarily re-enabling gradients via `torch.enable_grad()`. On XLA, mixing `no_grad` and `enable_grad` contexts can cause graph inconsistencies.
3. Each Grad-CAM call would trigger a full XLA graph recompilation, making inference extremely slow.

The HoughCircles alternative is deterministic, fast, requires no gradients, and is fully XLA-traceable. It is less targeted than Grad-CAM (it finds the rim, not the specific defect location) but works reliably on TPU.

11. Multi-Process Entry Point — xmp.spawn

This is the biggest structural addition in the TPU notebook. The GPU notebook trains in a single process on a single device. The TPU notebook spawns 8 separate worker processes — one per chip — that train in parallel and synchronise their gradients.

The worker function

`_train_worker(rank, args, all_samples, assets)` is the function that each of the 8 chips runs independently. Inside it:

- `device = xm.xla_device()` assigns this worker to its chip
- `xm.is_master_ordinal()` checks if this is chip 0 (the one that prints and saves)
- The k-fold training loop runs as normal, but each fold uses a `DistributedSampler`
- `xm.rendezvous('fold_N_done')` acts as a barrier — all 8 chips wait here until every chip has finished the fold before moving to the next one

The entry point

```
xmp.spawn(_train_worker, args=(args, all_samples, assets), nprocs=get_num_tpu_chips(),
start_method='spawn')
```

`xmp.spawn` launches exactly `nprocs` copies of `_train_worker` (8 on a v5e-8), assigns each a unique rank (0-7), and manages the communication backend. It is the TPU equivalent of `torch.multiprocessing.spawn`.

Gradient synchronisation

The magic happens inside `xm.optimizer_step()`. When called, it:

1. Flushes the pending XLA graph (`mark_step`)
2. All-reduces gradients across all 8 chips (averages them)
3. Updates the model weights

The result is as if you had trained on a batch of $8 \times 192 = 1,536$ images at once, but the computation was distributed across 8 chips simultaneously.

Only chip 0 (`is_master_ordinal()`) prints logs and saves checkpoints. All chips participate in training and gradient synchronisation.

12. Complete Side-by-Side Hyperparameter Reference

All configurable values across both notebooks.

Parameter / Feature	GPU Notebook (v4.0)	TPU Notebook
--batch	2	192
--lr	5e-5	5e-5
--img-size	384	384
--accum-steps	32	1
--label-smooth	0.02	0.01
--mixup-alpha	0.10	0.20
--focal-gamma	2.0	2.5
--ema-decay	0.9998	0.9999
--grad-clip	0.5	0.5
--dropout-rate	0.35	0.35
--val-split	0.12	0.12
--held-out-frac	0.05	0.05
--phase1-epochs	5	5
--phase2-epochs	8	8
--phase3-epochs	27	27
--swa-start	30	30
--early-stop	12	12
--folds	5	5
--bf16	False	True (always on)
--channels-last	True	False
--compile	False	False
--grad-checkpoint	True	False
--workers	2	4
STAGE1_BATCH	8	128
STAGE1_EPOCHS	15	15

Parameter / Feature	GPU Notebook (v4.0)	TPU Notebook
STAGE1_THRESHOLD	0.70	0.70
STAGE1_IMG_SIZE	384	384
Augmentation: GridDistortion	Absent	A.GridDistortion(p=0.25)
Augmentation: RandomShadow	Absent	A.RandomShadow(p=0.20)
Rim crop method	Fixed relative coords	HoughCircles adaptive
Grad-CAM crop	Present (inference)	Removed (XLA incompatible)
DistributedSampler	Not used	Used (8-chip distribution)
MpDeviceLoader	Not used	All loaders wrapped
xm.mark_step()	Not used	After every batch/EMA/save
xm.optimizer_step()	Not used	Replaces optimizer.step()
xm.save()	Not used	Replaces torch.save()
xmp.spawn()	Not used	Entry point (8 processes)

End of Guide
You now know every difference between the GPU (v4.0) and TPU notebooks.